A Comparative Survey of Minimum Spanning Tree Algorithms: Classical Approaches, Recent Improvements, and Real-World Applications

Md. Samiel Islam Sami

***Abstract*—This paper presents a comparative survey of classical MST algorithms, including Kruskal's, Prim's, and Borůvka's algorithms, highlighting their structure and practical characteristics. The survey also introduces recent developments in MST, addressing the limitations of classical approaches, including parallel, distributed, and dynamic large-scale MST computation. The paper also showcases the real-world applications and usage of MST. Overall, the survey highlights the evolution of MST algorithms from classical greedy methods toward modern application-specific computational environments.**

***Index Terms*—MST, Graph Algorithms, Greedy Algorithms, Parallel Computing, Dynamic Graphs.**

I. Introduction

The Minimum Spanning Tree (MST) problem is a fundamental problem in graph theory. A spanning tree is defined as a tree-like subgraph of a connected, undirected graph that includes all the vertices of the graph. Meanwhile, the minimum spanning tree includes only the minimum possible weights among all possible spanning trees. Like a spanning tree, there can also be many possible MSTs for a graph. MST problem has played an important role in the development of efficient algorithms. It is essential in computer science, communication and network problems, transportation systems, and power distribution.

One of the earliest solutions for the MST problem was proposed by Borůvka in 1926. Later, his work was followed by the most notable Kruskal's algorithm and Prim's algorithm. Kruskal's algorithm follows a greedy strategy, while Prim's algorithm expands a tree from a selected starting vertex. While, Borůvka's algorithm repeatedly connects components using their minimum-weight outgoing edges. Traditional sequential algorithms may become inefficient when processing graphs containing a large number of vertices and edges. Furthermore, many real-world networks are dynamic; edges may be inserted, deleted, and their weights can be modified. Such situations motivate research into dynamic MST algorithms [1]. The theoretical development of MST algorithms has also produced increasingly sophisticated techniques with improved asymptotic running times. However, an algorithm with a better theoretical complexity is not necessarily the best choice in practical applications.

Classical algorithms such as Kruskal's and Prim's remain the most popular choice because of their simplicity. Recently, many sophisticated techniques have been developed for MST problems, but they haven't been the best choice in practical applications. This research presents a comparative survey of Minimum Spanning Tree algorithms, with emphasis on the evolution from classical approaches to recent approaches. First, the fundamental concepts of MSTs are introduced. Alongside, classical algorithms of Borůvka, Kruskal, and Prim are discussed and compared. Subsequently, recent developments in areas such as dynamic, parallel, and large-scale MST computation are examined. The paper also shows the real-world applications of MST techniques and discusses the challenges and limitations faced when applying these algorithms to modern network environments.

II. Problem Statement and Research Objectives

Although classical algorithms provide efficient solutions to the Minimum Spanning Tree (MST) problem, their practical limitations become increasingly significant as graph size and computational requirements grow. Modern applications and uses may involve graphs containing millions or billions of edges, vertices, and rapidly changing network structures. Recent MST research has focused less on replacing the fundamental greedy principle and more on improving how MST computation is performed in large-scale, dynamic, and particular scenarios. The primary objectives of this survey are:

1) To review the classical Minimum Spanning Tree algorithms for MST problem.
2) To compare the performance, strengths, and limitations of Kruskal's, Prim's, and Borůvka's algorithms
3) To examine recent improvements in MST computation in parallel, dynamic, and large-scale graph environments.
4) To analyze how recent approaches address performance limitations of classical MST algorithms.
5) To review important real-world applications of MST algorithms in networking, transportation, clustering, and infrastructure design.

III. CLASSICAL MST ALGORITHMS

The development of Minimum Spanning Tree algorithms has introduced several greedy approaches. These algorithms mainly differ in how they select edges when creating the spanning tree. This research paper covers the classical algorithms Borůvka's algorithm, Kruskal's algorithm, and Prim's algorithm. These algorithms continue to be used as building blocks for many advanced MST approaches over the years.

A. Borůvka's Algorithm

Borůvka's algorithm is one of the earliest solutions to the Minimum Spanning Tree (MST) problem. It was developed by Otakar Borůvka in 1926, originally to solve the problem of constructing a sufficient electrical network. The algorithm first treats every vertex as a separate component. For each component, the minimum-weight edge connecting that component with another component is identified. This way, all selected edges are then added to the spanning forest. The process repeats itself until only one component remains. The main steps can be summarized as follows:

1) Start with each vertex as an independent component.
2) For every component, find its cheapest outgoing edge.
3) Add the selected edges to the forest.
4) Merge the connected components.
5) Repeat until a single component remains.

The component-based nature of Borůvka's algorithm makes each phase to be carried out independently, making this technique really crucial. Under standard implementations, Borůvka's algorithm requires $O(E \log V)$ time complexity. Its major advantage is its suitability for parallel processing. Meanwhile, one of the major disadvantages is that the implementation can be more complicated than the simpler classical approaches.

B. Kruskal's Algorithm

Kruskal's algorithm was created by Joseph B. Kruskal in 1956. Unlike Borůvka's component-based approach, Kruskal's algorithm processes edges globally according to their weight values. The algorithm first sorts all edge weights in increasing order. Then, it checks the edges one at a time and adds an edge to the MST, while also ensuring that it does not create a cycle. This process continues until all vertices are connected. Here, the main steps are:

1) Sort all edges in ascending order of weight.
2) Start with an empty set of selected edges.
3) Select the next lightest edge.
4) Add the edge if it does not form a cycle.
5) Continue until $(V - 1)$ edges have been selected.

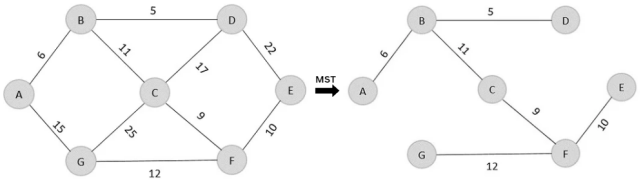


Fig. 1. Kruskal's Algorithms Example

To ensure efficient implementations, generally the Disjoint Set Union or Union-Find data structure is used to determine whether two vertices already belong to the same component. This avoids searching the entire graph to identify a cycle after every edge selection. The main operation in the standard Kruskal's algorithm is sorting the edges, resulting in a time complexity of approximately $O(E \log E)$, where E is the number of edges. Kruskal's algorithm is particularly popular for sparse graphs and is relatively straightforward to implement. The main downside is that the requirement to sort all edges can create additional overhead for very large graphs.

C. Prim's Algorithm

Prim's algorithm was introduced by Robert C. Prim in 1957. The algorithm builds the MST gradually from an initial vertex rather than processing all edges globally. First, Prim's algorithm starts from any vertex and repeatedly selects the minimum-weight edge connecting the current tree to an unvisited vertex. The selected vertex is then added to the tree, and the process continues until all vertices have been included. Here, the main steps are:

1) Select an arbitrary starting vertex.
2) Identify the minimum-weight edge connecting the current tree to an unvisited vertex.
3) Add that edge and vertex to the tree.
4) Update the candidate edges.
5) Repeat until all vertices are included.

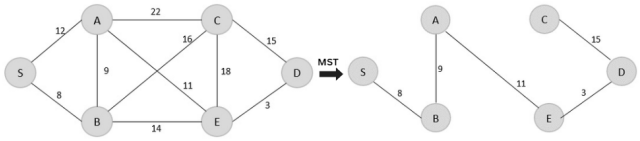


Fig. 2. Prim's Algorithms Example

In Prim's algorithm, a priority queue is generally used to identify the minimum-weight candidate edge. With an adjacency-list representation and a binary heap, the time complexity of Prim's algorithm is $O((V + E) \log V)$. In best case scenerios, a standard Prim's algorithm implementation achieves $O(E \log V)$ time complexity. For dense graphs, an array-based implementation using an adjacency matrix can be used. When adjacency matrix is used the time complexity is $O(V^2)$.

One of the main advantages of Prim's algorithm is that it can directly grow a connected tree and also perform well when the graph is represented using adjacency structures. However, its performance depends significantly on the graph representation and data structures.

D. Comparison of Classical Algorithms

Although all three algorithms produce an MST using greedy principles, their strategies are fundamentally different. Borůvka's algorithm operates on components, Kruskal's algorithm operates on sorted edges, and Prim's algorithm grows a single tree from a starting vertex.

In practical scenarios, factors such as graph density, data structure, memory organization, sorting overhead, and hardware architecture can significantly influence execution time. Consequently, real-time comparison is important when determining which algorithm is most appropriate for a particular

Algorithm	Selection Strategy	Key Data Structure	Advantage
Borůvka	Cheapest outgoing edge of each component	Component representation	Parallel/distributed processing
Kruskal	Globally lightest available edge	Union-Find	Simple and effective for sparse graphs
Prim	Cheapest edge connecting tree to outside vertex	Priority Queue	Effective for connected graphss

TABLE I
COMPARISON OF CLASSICAL MST ALGORITHMS

application [1]. The classical algorithms also provide the foundation for modern MST research. Borůvka's component-based strategy is specially suitable for parallelization. On the other hand, Kruskal's edge-sorting approach is useful in large-scale graph processing. Meanwhile. Prim's algorithm is also important for sequential implementations.

IV. RECENT IMPROVEMENTS

Recent developments can be categorized into the following cases: 1) Massively parallel and distributed MST algorithms, 2) Dynamic and incremental MST algorithms, 3) MST computation in metric spaces and approximate settings, and 4) Hardware-specific approaches. These developments are made exclusively for finding an optimal tree, while also keeping the computational constraints in check.

A. Massively Parallel and Distributed MST Algorithms

One of the major limitations of classical MST algorithms is that their standard implementations are primarily designed for sequential execution. For example, Kruskal's algorithm requires global edge sorting. While, Prim's algorithm repeatedly selects the minimum-weight edge from the current tree. Such dependencies can limit the amount of parallelism that can be utilized. On the other hand, Borůvka's algorithm provides a more suitable architecture for parallel computation because each component can independently identify its cheapest outgoing edge. For this reason, research into parallel and distributed MST algorithms is essential.

The research [2], worked on practical parallel implementations of Borůvka's algorithm on multicore processors. Rather than focusing only on theoretical parallel models. This study addressed limitations encountered on real hardware, including memory bandwidth and the practical number of available processors. Two implementations were created using synchronization mechanisms that are based on atomic compare-and-swap operations and lock variables. Experimenting on graphs that contained up to one million vertices showed speedups over sequential implementations. This proves that Borůvka's component-based structure can be effectively exploited on multicore systems.

A more recent and larger-scale contribution is the work of Sanders and Schimek [3]. Their work proposes distributed-memory systems rather than merely multicore execution. They also developed scalable MST algorithms based on Borůvka's architecture. Additionally, they introduced two important practical improvements. First, for partitioned graphs with many edges, they performed a preprocessing step that contracts local portions of the graph. Doing this reduces the amount of data that was to be processed across the distributed system. Second, they introduce a Filter-Borůvka approach that introduces a filtering technique to reduce unnecessary edges before performing the main MST computation. This is particularly useful for graphs with high average degree and poor locality. The experimental results in this paper is significant for the future of MST algorithms. Their implementations scale to around 65,536 processor cores. They are also reported to achieve 800 times speedups compared with previous distributed MST implementations.

B. Dynamic and Incremental Minimum Spanning Trees

Classical MST algorithms usually assume that the input graph remains unchanged during computation. However, we know that many real-world networks are dynamic. For example, road networks may change, communication links may fail or be added, and temporal graphs may receive new edges. In such cases, computing an MST from scratch after every modification can be unnecessarily expensive and time-consuming. This is the exact reason many researchers started working on dynamic MST algorithms, which attempt to maintain an MST even if the graph changes.

In the paper [4], Anderson, Blelloch, and Tangwongsan studied batch-incremental MST computation, which focuses on situations in which multiple edges are inserted into a graph. The authors developed an efficient parallel algorithm for incremental MSTs in which the processing cost depends on the number of inserted edges rather than computing the whole graph each time. The compressed path tree is a key component of this research paper, which captures the relevant maximum-weight relationships between selected vertices in an existing edge-weighted tree. By using this technique, the algorithm can efficiently determine how new edges affect the existing MST. The paper also showed approximate MSTs, bipartiteness testing, graph connectivity, sliding-window problems, and graph sparsification.

On the other hand, the paper [5] examined incremental MST algorithms in the context of temporal graphs. Classical structures provide strong theoretical guarantees but may suffer from substantial practical overhead. To address this issue, the authors introduced a new data structure called the AM-tree. The AM-tree is designed to retain theoretical guarantees compared to link-cut-tree-based approaches while improving practical usage. Their experiments showed that the AM-tree can be approximately 7.8–11 times faster for updates and 7.7–13.7 times faster for queries than link-cut-tree-based implementations.

C. Approximate and Learning-Augmented MST Algorithms

A recent development is the use of approximation and data-driven techniques for MST problems in metric spaces.

The paper [6] studied MST computation for (n) points in an arbitrary metric space. The authors proposed a framework that constructs a forest using practical heuristics and then identifies a relatively small set of edges capable of connecting the resulting components. They managed to develop a subquadratic 2.62-approximation algorithm. Also, the work introduces a learning-augmented perspective. Their experiments produced nearly optimal spanning trees for a range of metrics while being substantially faster. This direction is significant for applications involving machine learning and high-dimensional data.

D. Hardware-Specific and Emerging Approaches

MST computation is also being investigated on specialized hardware architectures. One recent example is the work [7], where the researchers explore a neuromorphic implementation of Kruskal's algorithm. Neuromorphic systems use event-driven computation and massive parallelism, providing a different computational environment from conventional CPUs. This research identified an important bottleneck in the conventional Kruskal's algorithm. Sorting and Union-Find operations are largely separate sequential stages. They propose an approach where neuromorphic sorting and Union-Find operations can be executed at the same time using spike-driven computation. Experiments in this paper showed substantial speedups up to approximately 540 times compared with the Prim-based methods. They also report significant improvements over sequential variants of Kruskal's algorithm. Although neuromorphic MST computation is an emerging concept, it demonstrates promising outcomes.

V. Real-World Application

Minimum Spanning Tree algorithms have a wide range of usage in network design and data analysis problems. Some of MST's real-world usages are:

1) Computer and Communication Networks: MST algorithms can be used to design cost-efficient communication networks. This is used particularly in designing wired networks, where unnecessary networks and connections can increase infrastructure and maintenance costs. Also, MST-based approaches can work in network clustering and topology optimization.

2) Transportation and Infrastructure: In transportation networks, vertices can represent locations such as cities or stations, while edge weights can represent construction costs or distances. An MST can identify the best possible set of connections that links all required locations. This principle can be applied for efficient planning of infrastructure such as roads, railway connections, and utility networks.

3) Network and Power: MST techniques are relevant to network planning problems where multiple locations must be connected while minimizing the cost of infrastructure. In such applications, additional constraints such as capacity and redundancy are considered alongside the basic MST objective.

4) Image Processing: MST-based techniques can be used in image segmentation, where pixels or image regions are represented as vertices and the edge weights represent differences between neighboring pixels. By constructing an MST, image regions with similar characteristics can be connected, while larger-weight edges can be used to identify boundaries between different regions.

5) Clustering and Data Analysis: MSTs are used in data analysis and clustering. Data can be represented as vertices, with edge weights representing a distance or dissimilarity value between objects. MST provides relationships among the data points. Expensive edges can be removed to separate the tree into groups. As a result, MST-based methods are useful in hierarchical and distance-based clustering.

VI. Conclusion

This survey examined the development of MST algorithms from classical greedy approaches to recent research addressing the challenges of modern large-scale graph processing. Though the classical algorithms exploits the greedy selection they fall short in certain scenarios. Recent research has therefore worked on improving the said limitations and shortcomings of the general MST algorithms. Modern MST research is concerned with scalability, adaptability, and practical performance. Also, MST techniques are commonly used in communication networks, transportation and infrastructure planning, networks, image processing, and many others. Future research should therefore focus on developing MST algorithms that can efficiently handle increasingly large and continuously changing graphs. Combining MST computation with approximation, machine learning, and hardware-aware optimization also showed promise. Ultimately, the evolution of MST algorithms depends on the adaptability, complexity, and computational environments of modern applications.

References

[1] C. Bazlamaçı and S. Hindi Khalil, "Minimum-weight spanning tree algorithms: A survey and empirical study," *Computers & Operations Research*, vol. 28, no. 8, pp. 767–785, 2001. [Online]. Available: https://doi.org/10.1016/S0305-0548(00)00007-1

[2] S. M. Durbhakula, "Parallel minimum spanning tree algorithms and evaluation," *arXiv preprint arXiv:2005.06913*, 2020. [Online]. Available: https://arxiv.org/abs/2005.06913v1

[3] P. Sanders and M. Schimek, "Engineering massively parallel mst algorithms," *arXiv preprint arXiv:2302.12199*, 2023. [Online]. Available: https://arxiv.org/abs/2302.12199v2

[4] D. Anderson, G. E. Blelloch, and K. Tangwongsan, "Work-efficient batch-incremental minimum spanning trees with applications to the sliding window model," *arXiv preprint arXiv:2002.05710*, 2020. [Online]. Available: https://doi.org/10.1145/3350755.3400241

[5] X. Ding, Y. Gu, and Y. Sun, "New algorithms for incremental minimum spanning trees and temporal graph applications," *arXiv preprint arXiv:2504.04619*, 2025. [Online]. Available: https://arxiv.org/abs/2504.04619v2

[6] N. Veldt, T. Stanley, B. W. Priest, T. Steil, K. Iwabuchi, T. S. Jayram, and P. Sanders, "Approximate tree completion and learning-augmented algorithms for metric minimum spanning trees," *arXiv preprint arXiv:2502.12993*, 2025. [Online]. Available: https://arxiv.org/abs/2502.12993v1

[7] H. Chong, P. Qu, Y. Li, and Y. Zhang, "Pipelining kruskal's: A neuromorphic approach for minimum spanning tree," *arXiv preprint arXiv:2505.10771*, 2025. [Online]. Available: https://arxiv.org/abs/2505.10771v2